COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/04/2026	**DATE SUBMITTED:**	09/07/2026
NAME:	Moradillo, Eunice E.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 7.1 Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
#include <iostream>
#include <cstdlib>
#include<ctime>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);



int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize];
    //use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100; //random values between 0 and 99
    }
    //display the randomly generated array
    std::cout<<"unsorted array: "<<std::endl;
    displayArr(unsortedArr);

    //std::cout<<"bubble sort: "<<std::endl;
    //bubbbleSort(unsortedArr, maxSize);
    //displayArr(unsortedArr);

    //std::cout<<"selection sort: "<<std::endl;
    //selectionSort(unsortedArr, maxSize);
    //displayArr(unsortedArr);

   // std::cout<<"insertion sort: "<<std::endl;
    //insertionSort(unsortedArr, maxSize);
   // displayArr(unsortedArr);


}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

}
```

```
unsorted array:
36 67 45 11 27 68 72 56 14 1 66 21 4 70 34 15 3 67 25 22 85 99 55 76 38 59 14 64 73 58 25 1 19 8 78 1 5 87 87 76 30 18 46 8 64 32 91 39 65 55 46
 38 45 86 25 71 46 75 70 66 60 37 53 88 66 77 48 70 85 83 12 84 31 80 0 31 71 34 40 47 3 11 3 77 38 73 91 70 23 1 37 4 82 99 68 49 21 61 7 48
```

Observations	The array successfully generated with 100 random integer values ranging from 0 to 99 using rand(), seeded with srand(time(0)) for a different sequence each run. The unsorted array displayed exactly 100 values with no visible ordering pattern, confirming the random generation worked correctly before sorting.

Table 7-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	
Observations	The bubble sort correctly arranged the 100 numbers from smallest to largest by repeatedly comparing pairs of numbers next to each other and swapping them when they were out of order, taking O(n²) time since the code did not stop early even when the array became sorted. This made it do the most swaps out of the three sorts, but it still only needed O(1) extra space since it sorted the array in place.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

```
std::cout<<"selection sort: "<<std::endl;
selectionSort(unsortedArr, maxSize);
displayArr(unsortedArr);
```

```
template <typename T>
//array, current position, size of array
int routineSmallest(T arr[], int K, const int arrSize){
    int position;
    T smallestElem = arr[K];

  for (int j = K+1; j< arrSize; j++){
    if(arr[j] < smallestElem){
        smallestElem = arr[j];
        position = j;
    }
  }
  return position;
}

template <typename T>
void selectionSort(T arr[], size_t arrSize){
    int POS, temp, pass;
    for (int i= 0; i < arrSize; i++){
        POS = routineSmallest(arr, i, arrSize);
        //swapping:
        temp = arr[i];
        arr[i] = arr[POS];
        arr[POS] = temp;
        //for verification and counting of number of passes that was done
        pass++;
    }

}
```

```
unsorted array:
17 97 56 16 14 73 4 81 2 58 76 24 44 66 0 15 39 32 75 80 48 24 44 52 43 57 37 6 20 44 65 70 34 76 88 5 0 76 60 34 88 84 99 21 8 76 38 63 27
5 39 27 84 22 94 52 33 38 33 26 75 66 78 94 99 82 5 48 80 39 58 90 68 52 32 3 24 93 11 77 39 87 44 32 89 25 82 15 65 26 63 27 52 92 99 47 53
 12 71 1
selection sort:
0 0 1 2 3 4 5 5 5 6 8 11 12 14 15 15 16 17 20 21 22 48 24 24 24 25 26 26 27 27 27 32 32 32 33 33 34 34 37 38 38 39 39 39 43 44 44 44 44 4
7 48 52 52 52 52 53 56 57 58 58 60 63 63 65 65 66 66 68 70 71 73 75 75 76 76 76 76 77 78 80 80 81 82 82 84 84 87 88 88 89 90 92 93 94 94 97
99 99 99
```

Observations

The selection sort correctly sorted the array by finding the smallest number left in the unsorted part and swapping it into place, one number at a time, which also took $O(n^2)$ time no matter how the numbers started out. It only made one swap per pass, so it needed fewer total swaps than bubble sort while still using just $O(1)$ extra space.

Table 7-3. Selection Sort Algorithm

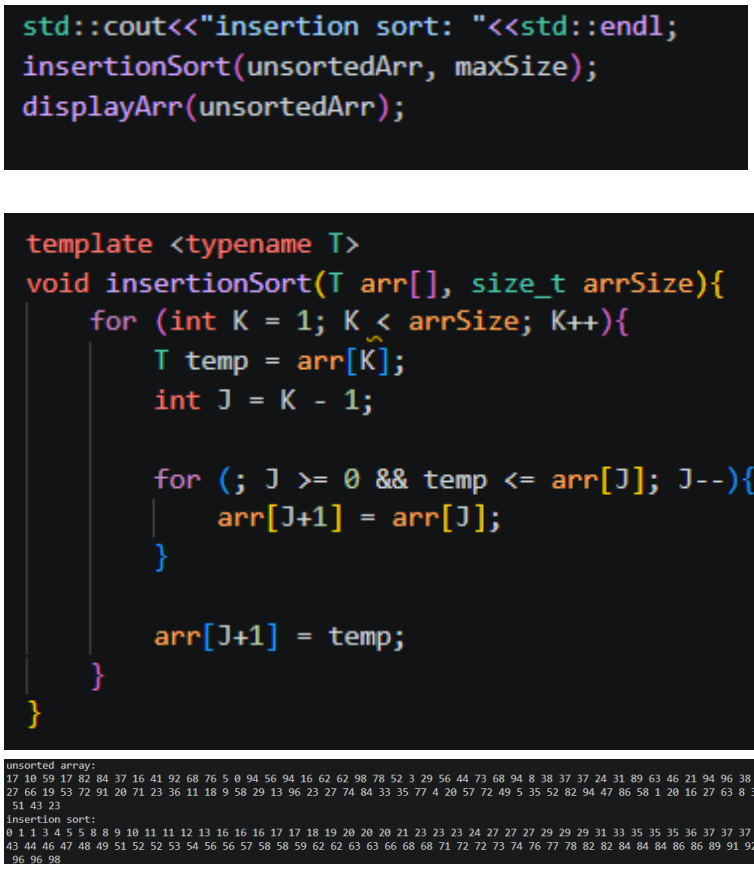
Code + Console Screenshot	
Observations	The insertion sort correctly sorted the array by taking each number and shifting the larger ones to the right until it found the right spot to insert it, which took O(n²) time since the starting array was fully random. It also used only O(1) extra space, and it would have run much faster, close to O(n), if the array had already been mostly sorted.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode of Algorithm	1. START 2. Generate an array of 100 random unsorted values 3. Display the unsorted array 4. Sort the array using bubble sort and display the result 5. Sort the array using selection sort and display the result 6. Sort the array using insertion sort and display the result 7. END
Screenshot of Algorithm Code	

```cpp
#include <iostream>
#include <cstdlib>
#include<ctime>
#include <string>
#include "sortingAlgo.h"

const int maxSize = 100;
void displayArr(int arr[]);



int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize];
    //use time (0) as seed for random values
    srand(time(0));
    //populate the array with random values
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand() % 100; //random values between 0 and 99
    }
    //display the randomly generated array
    std::cout<<"unsorted array: "<<std::endl;
    displayArr(unsortedArr);

   std::cout<<"bubble sort: "<<std::endl;
   bubbbleSort(unsortedArr, maxSize);
   displayArr(unsortedArr);

//  std::cout<<"selection sort: "<<std::endl;
//selectionSort(unsortedArr, maxSize);
// displayArr(unsortedArr);

  // std::cout<<"insertion sort: "<<std::endl;
//   insertionSort(unsortedArr, maxSize);
//   displayArr(unsortedArr);

  // Vote Counting
    const int numCandidates = 5;
    std::string candidateNames[numCandidates + 1] = {
        "", //index 0 unused, candidates are numbered 1-5
        "Bo Dalton Capistrano",
        "Cornelius Raymon Agustin",
        "Deja Jayla Banaga",
        "Lalla Brielle Yabut",
        "Franklin Relano Castro"
    };
```

```
    //generate an array of 100 votes, values ranging from 1 to 5
    int votesArr[maxSize];
    for(int i = 0; i < maxSize; i++){
        votesArr[i] = (rand() % numCandidates) + 1; //random values between 1 and 5
    }

    std::cout<<"\nunsorted votes array: "<<std::endl;
    displayArr(votesArr);

    //sort the votes using insertion sort (prints number of passes internally)
    insertionSort(votesArr, maxSize);

    std::cout<<"sorted votes array: "<<std::endl;
    displayArr(votesArr);

    //since the array is sorted, tally each candidate's votes in a single pass
    int voteTally[numCandidates + 1] = {0};
    for(int i = 0; i < maxSize; i++){
        voteTally[votesArr[i]]++;
    }

    //determine the winning candidate
    int winner = 1;
    for(int c = 2; c <= numCandidates; c++){
        if(voteTally[c] > voteTally[winner]){
            winner = c;
        }
    }

    std::cout<<"\nvote count result: "<<std::endl;
    for(int c = 1; c <= numCandidates; c++){
        std::cout<<"Candidate "<<c<<" ("<<candidateNames[c]<<"): "<<voteTally[c]<<" votes"<<std::endl;
    }
    std::cout<<"\nwinning candidate: Candidate "<<winner<<" ("<<candidateNames[winner]<<") with "<<voteTally[winner]<<" votes"<<std::endl;

}

void displayArr(int arr[]){
    for (int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

}
```

Output Testing

BUBBLE SORT

```
unsorted array:
67 86 34 58 71 45 82 99 63 53 91 34 90 28 87 42 38 66 8 51 43 87 17 77 58 17 96 46 33 30 80 28 69 48 25 18 59 96 22 21 71 92 39 9 82 88 59 4
4 95 62 23 57 28 45 27 18 34 15 12 13 91 35 39 99 6 58 93 48 20 51 60 42 45 80 25 1 16 99 54 72 88 55 38 66 47 44 23 37 64 12 47 3 96 30 70
62 68 47 74 93
bubble sort:
1 3 6 8 9 12 12 13 15 16 17 17 18 18 20 21 22 23 23 25 25 27 28 28 28 30 30 33 34 34 34 35 37 38 38 39 39 42 42 43 44 44 45 45 45 46 47 47 4
7 48 48 51 51 53 54 55 57 58 58 58 59 59 60 62 62 63 64 66 66 67 68 69 70 71 71 72 74 77 80 80 82 82 86 87 87 88 88 90 91 91 92 93 93 95 96
96 96 99 99 99

unsorted votes array:
2 1 5 3 1 4 5 4 4 1 2 1 2 4 2 3 4 2 5 4 4 3 3 3 2 3 5 2 1 5 2 3 2 4 3 1 1 3 4 1 1 1 4 1 4 5 4 5 2 3 1 1 2 5 4 1 1 2 1 4 5 3 1 5 4 3 4 2 3 3
4 3 2 2 2 1 1 4 3 4 4 1 1 1 3 2 4 4 1 5 2 2 2 3 3 5 2 4 1 2
number of passes: 99
sorted votes array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5

vote count result:
Candidate 1 (Bo Dalton Capistrano): 24 votes
Candidate 2 (Cornelius Raymon Agustin): 22 votes
Candidate 3 (Deja Jayla Banaga): 19 votes
Candidate 4 (Lalla Brielle Yabut): 23 votes
Candidate 5 (Franklin Relano Castro): 12 votes

winning candidate: Candidate 1 (Bo Dalton Capistrano) with 24 votes
```

SELECTION SORT

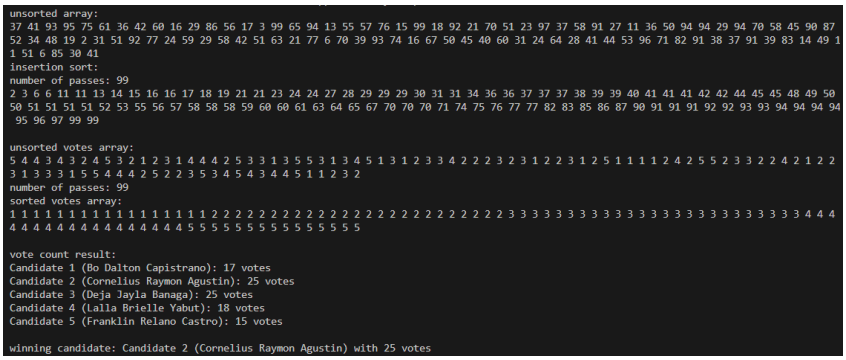
```
unsorted array:
91 96 41 91 40 78 50 32 73 68 78 56 7 30 26 89 59 44 53 70 30 16 16 5 30 66 14 95 1 49 1 24 57 73 73 0 53 6 31 81 19 30 89 38 94 7 4 63 90 6
4 9 89 74 70 4 12 13 41 82 68 45 79 50 21 80 95 19 90 43 47 19 52 91 4 63 2 85 22 81 28 2 34 86 73 4 90 54 13 76 17 72 51 0 3 31 21 75 72 9
13
selection sort:
0 0 1 1 2 2 3 4 4 4 4 5 6 7 7 9 9 12 13 13 13 14 16 16 17 19 19 19 21 21 22 91 24 26 28 30 30 30 30 31 31 32 34 89 38 40 41 41 43 44 45 47 4
9 50 50 51 52 53 53 54 56 57 59 70 63 63 64 66 68 68 70 72 72 73 73 73 73 74 75 76 80 78 78 79 81 81 82 85 95 86 91 89 89 90 90 90 91 96 94
95

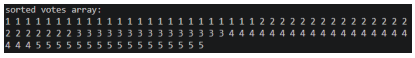
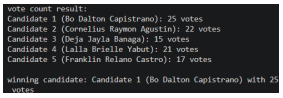
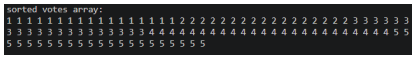
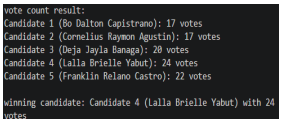
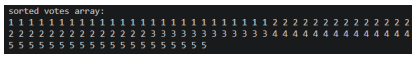
unsorted votes array:
5 2 3 3 5 1 3 3 4 1 5 4 5 5 4 4 3 2 3 1 3 3 2 1 5 3 3 5 4 5 2 2 4 3 2 1 2 4 2 4 5 3 5 1 3 3 5 5 5 3 2 2 1 2 1 5 2 2 5 1 5 4 1 3 4 1 3 5 2 5
2 2 4 5 3 5 1 1 2 1 1 5 2 2 5 3 5 5 2 4 2 4 2 3 1 2 3 2 5 1
number of passes: 99
sorted votes array:
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4
4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5

vote count result:
Candidate 1 (Bo Dalton Capistrano): 17 votes
Candidate 2 (Cornelius Raymon Agustin): 24 votes
Candidate 3 (Deja Jayla Banaga): 21 votes
Candidate 4 (Lalla Brielle Yabut): 13 votes
Candidate 5 (Franklin Relano Castro): 25 votes

winning candidate: Candidate 5 (Franklin Relano Castro) with 25 votes
```

INSERTION SORT



SORT	Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
BUBBLE		1: 25 votes 2: 22 votes 3: 15 votes 4: 21 votes 5: 17 votes Winner: 1(25 votes) Total : 100	
SELECTION		1: 17 votes 2: 17 votes 3: 20 votes 4: 24 votes 5: 22 votes Winner: 4(24 votes) Total : 100	
INSERTION		1: 26 votes 2: 28 votes 3: 12 votes 4: 14 votes 5: 20 votes Winner: 2(28 votes) Total : 100	

Best Sorting Algorithm and justify:

I think that insertion sort is the best pick for this vote counting problem because the array only has 5 different values (1 to 5) repeating many times, so insertion sort does not need to shift numbers very far to put them in the right spot. Bubble sort is less efficient here since it always checks every pair of numbers next to each other on every pass, even when many of them are already the same value. Selection sort also works but still has to scan the whole

unsorted part of the array every single pass, so insertion sort ends up being the more practical choice since the data has a lot of repeated values it can take advantage of.

Question: Was your developed vote counting algorithm effective? Why or why not?
The vote counting algorithm was effective. When I manually counted the votes for each candidate and compared it to the algorithm's output, the numbers matched exactly. This shows the algorithm correctly counted the votes, since all three sorting algorithms are $O(n^2)$, they are effective for a small dataset like this (100 votes) but would become inefficient with a much larger number of voters.

CONCLUSION

This activity helped me understand how bubble sort, selection sort, and insertion sort work and how each one sorts a list in its own way, even though they all end up with the same correct result and share $O(n^2)$ time complexity with O(1) extra space, since none of them need any extra array to do the sorting. Bubble sort swaps numbers next to each other over and over across many passes, selection sort picks the smallest number left in the unsorted part and puts it in place one at a time, and insertion sort takes each number and slides it into the right spot among the numbers already sorted, and testing all three on a random 100-number array showed they all sorted correctly, just with different amounts of swapping and comparing along the way. Building these algorithms also helped me remember what our professor said, which let the same sort functions work no matter what type of data was being sorted, and organizing the code into a separate header file made the program easier to read and manage. For the supplementary activity, I used insertion sort to arrange 100 votes (numbers 1 to 5) so that identical votes were grouped together, then counted how many times each number appeared to find the winning candidate, and comparing my manual count to the algorithm's output showed the numbers matched exactly, proving the vote counting was accurate and reliable.I think I did well in this activity since I was able to implement all three sorting algorithms correctly, understand the trade-offs between them, and apply one of them to solve a real counting problem, though I could improve by actually timing each algorithm or counting the real number of swaps and comparisons instead of just checking that the final sorted output looked correct, especially since these differences would matter more with a much larger dataset.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google bing ai**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **search**

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Use to expand the idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.